Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of Computer Science and Engineering

2025-2026
Odd Semester

DATA STRUCTURES AND ALGORITHMS-3 (25CS2103E)

Tree Diameter Computation System

Using Tree Dynamic Programming and Advanced Algorithms

P. Nakshatra	**2520030243**
Ch. Navya Sai Sree	**2520030244**
B. Himavanth Sai Ganesh	**2520030429**

COURSE INSTRUCTOR

Dr. Swathi
Professor
Department of Computer Science and Engineering

CONTENTS

No.	Section
	Part A: Project Report
1	Cover Page
2	Introduction
3	Problem Statement
4	Objectives of the Project
5	Application of Advanced Algorithms
6	Practical Case Study Examples
7	Expected Results and Benefits
8	Conclusion
	Part B: Algorithm / Pseudocode
9	Introduction to the Algorithmic Workflow
10	Overall Algorithm of the Website
11	Tree Input and Validation Algorithm
12	Adjacency List Construction
13	DFS and Tree Dynamic Programming
14	Diameter Path Reconstruction
15	Tree Visualisation
16	KMP String Matching Algorithm
17	Edmonds-Karp Maximum-Flow Algorithm
18	Vertex Cover 2-Approximation Algorithm
19	Randomized Quicksort Algorithm
20	Complexity and Algorithm Analysis
21	Pseudocode Conclusion
	Part C: Results
22	Results and Website Screenshots
23	Final Conclusion

The cover page is section 1 of the report. Parts A, B, and C each begin on a new page.

PART A: PROJECT REPORT

2. Introduction

2.1 Trees and Graph Structures

A graph is a data structure made up of nodes (also called vertices) and edges that connect pairs of nodes. Graphs are used to model road maps, computer networks, social connections, and many other systems in which objects are related to one another. A tree is a special kind of graph. It is connected, which means that every node can be reached from every other node, and it contains no cycles. A tree with N nodes always has exactly N - 1 edges, and there is exactly one simple path between any two nodes.

Because of these properties, trees are used to represent file systems, organisation charts, expression structures, decision processes, and hierarchical data in general. Many problems that are difficult on general graphs become simpler on trees, and this makes trees an excellent starting point for studying graph algorithms.

2.2 Tree Diameter

The diameter of a tree is the length of the longest simple path between any two of its nodes. In this project, length is measured as the number of edges on the path. For example, in a chain of five nodes the diameter is 4, and in a star with one centre and several leaves the diameter is 2. The two end points of a diameter path are always leaf nodes (or, in a trivial tree, the only node itself).

The diameter is a useful measure of how spread out a tree is. In a network, it indicates the largest number of hops between two machines. In a hierarchy, it shows how far apart two members can be. A tree can have more than one longest path, so a program that reports the diameter must also state clearly which path it displays.

2.3 Advanced Algorithms

The course Data Structures and Algorithms-3 (Advanced Algorithms) studies techniques that go beyond basic searching and sorting. These include algorithm analysis with asymptotic notation, string-matching algorithms, dynamic programming, network-flow algorithms, approximation algorithms, and randomised algorithms. Each technique addresses a different style of problem, and each can be demonstrated on a tree.

The Tree Diameter Computation System combines these ideas in one website. Its main algorithm uses Tree Dynamic Programming together with Depth First Search (DFS) to compute the diameter. Five further modules demonstrate algorithm analysis, Knuth-Morris-Pratt (KMP) string matching, Edmonds-Karp maximum flow, vertex cover 2-approximation, and randomised quicksort, all applied to the tree entered by the user.

2.4 Importance of Visualising Algorithmic Results

Algorithms are often learnt from pseudocode and numerical output alone, which can make it hard to see why an answer is correct. A drawing of the tree with the longest path highlighted lets the user verify the result at a glance. Visual feedback also helps to detect input mistakes, such as a missing edge, because the shape of the drawn tree will look different from what the user intended. For this reason, visualisation is an important part of the project and not merely a decoration.

2.5 About the Project

The Tree Diameter Computation System is a frontend-only website developed using HTML, CSS, and Vanilla JavaScript. The user enters the number of nodes and the list of edges. The website validates the input, builds an adjacency list, calculates the diameter, reconstructs the longest path, draws the tree with the diameter path highlighted in red, and displays the diameter, longest path, number of nodes, number of edges, time complexity, and space complexity. All calculations run directly in the browser. The project does not use a backend, a database, React, or Node.js.

3. Problem Statement

Finding the longest path in a tree by hand is manageable only when the tree is very small. For a tree with ten nodes, a student can trace paths on paper. For a tree with hundreds or thousands of nodes, the number of possible node pairs grows rapidly, and it becomes easy to miss a branch, miscount edges, or overlook a second path of equal length. A tree with N nodes has N(N - 1)/2 node pairs, so checking every pair manually is not practical.

A second difficulty is input correctness. A set of edges only describes a tree if the number of edges is exactly N - 1 and every node is connected to the rest. Typing errors such as a repeated edge, a node number outside the valid range, or a missing edge produce a structure that is not a tree, and any diameter computed from it would be meaningless.

A third difficulty lies in learning. Students who read about dynamic programming, KMP, network flow, approximation, and randomisation often find these topics abstract because they are presented separately and without a common example. There is a need for a tool that applies several advanced algorithms to the same input and shows the results visually.

The Tree Diameter Computation System addresses these problems. It validates the input, computes the diameter automatically in linear time, shows the longest path on a drawing of the tree, and demonstrates five additional advanced-algorithm modules on the same tree.

4. Objectives of the Project

The main objectives of the Tree Diameter Computation System are:

1. **Accepting Tree Input:** Allow the user to enter the number of nodes and the edges that connect them.
2. **Validating Nodes and Edges:** Check that the entered values are valid, that exactly N - 1 edges are given, and that the nodes form one connected structure.
3. **Adjacency-List Representation:** Store the validated tree as an adjacency list so that neighbours can be accessed efficiently.
4. **Efficient Diameter Calculation:** Calculate the diameter in O(N) time using Tree Dynamic Programming with DFS.
5. **Diameter Path Reconstruction:** Recover the actual sequence of nodes along the longest path, not only its length.
6. **Tree Visualisation:** Draw the tree on the webpage so that its structure is easy to understand.
7. **Highlighting the Diameter:** Display the diameter path in red so that it stands out from the rest of the tree.

8. **Demonstrating Advanced Algorithms:** Show algorithm analysis, KMP string matching, Edmonds-Karp maximum flow, vertex cover 2-approximation, and randomised quicksort on the entered tree.
9. **Displaying Complexity Information:** Present the time and space complexity along with the results.

4.1 Scope and Technology

Aspect	Description
Type of application	Frontend-only website that runs entirely in the web browser
Technologies used	HTML (structure), CSS (styling), Vanilla JavaScript (logic)
Technologies not used	Backend server, database, React, Node.js
Input	Number of nodes, list of edges; for specific modules, a numeric pattern and source and destination nodes
Output	Diameter, longest path, node count, edge count, complexity details, tree drawing, and module results
Main algorithm	Tree Dynamic Programming with DFS, O(N) time and O(N) space

5. Application of Advanced Algorithms

The website contains six algorithm modules. This section describes the purpose, working principle, reason for inclusion, relation to the tree, an example, and the complexity of each. Unless stated otherwise, N denotes the number of nodes, and a tree has N - 1 edges. The examples use the primary test case introduced in Section 6: a tree with 10 nodes and the edges 1-2, 1-3, 2-4, 2-5, 3-6, 3-7, 6-8, 6-9, and 7-10.

5.1 Module 1: Algorithm Analysis and Complexity

Purpose. This module explains how an appropriate algorithm is selected for the diameter problem and how its running time and memory use are described with asymptotic notation.

Working principle. Algorithm analysis estimates how the time and memory requirements grow with the input size. Big-O notation describes an upper bound on this growth. A simple approach to the diameter problem is to run a traversal from every node and record the longest distance found. Each traversal costs O(N), and N traversals cost O(N^2). A better approach, Tree Dynamic Programming with DFS, visits each node and each edge a constant number of times and therefore requires only O(N) time.

Why it is included. The project is about computing the diameter efficiently. Without analysis, there is no basis for choosing the dynamic-programming method over the simpler quadratic method, and the displayed time and space complexity would have no explanation.

Relation to the tree. A tree has only N - 1 edges, so its adjacency list uses O(N) memory. Every node's subtree is independent of its sibling subtrees, which is what allows a single traversal to solve the problem.

Example. For a chain of 100,000 nodes, a quadratic method performs roughly 10^10 basic steps, whereas the linear method performs about 10^5 steps. The difference shows why the choice of algorithm matters for large inputs.

Complexity. The main diameter algorithm runs in O(N) time and O(N) space. Complexities of the other modules are given in their own sections and summarised in Section 20.

5.2 Module 2: KMP String Matching

Purpose. This module searches for a numeric pattern in the DFS preorder traversal sequence of the tree. The Knuth-Morris-Pratt (KMP) algorithm finds every occurrence of a pattern in a sequence without re-examining characters that have already been matched.

Working principle. KMP first builds a failure table (also called the LPS array, for longest proper prefix which is also a suffix) for the pattern. When a mismatch occurs, the table tells the algorithm how far the pattern pointer can fall back without moving the text pointer backwards. The text pointer therefore only moves forward, which gives linear running time.

Why it is included. KMP is the string-matching technique studied in the course. A preorder traversal turns the tree into a sequence of node labels, so the same algorithm used for text can be applied to a tree-derived sequence.

Relation to the tree. A DFS preorder visits a node first and then its subtrees in order. For the primary test case, starting from node 1 and visiting neighbours in the order in which they appear in the adjacency list, the preorder sequence is 1, 2, 4, 5, 3, 6, 8, 9, 7, 10. A pattern is searched for within this sequence of node numbers.

Example. Searching for the pattern [3, 6, 8] in the sequence above finds one occurrence, starting at index 4 (the fifth element, using zero-based indexing). The pattern [2, 4, 5] is found at index 1. A pattern such as [4, 3] is not found, because node 4 is never directly followed by node 3 in this sequence.

Complexity. If the sequence has N elements and the pattern has M elements, building the failure table takes O(M) time and searching takes O(N) time, so the worst-case total is O(N + M). The extra memory is O(M) for the failure table.

5.3 Module 3: Tree Diameter Using Dynamic Programming

Purpose. This is the main algorithm of the project. It computes the diameter of the tree and reconstructs one longest path.

Working principle. The tree is rooted at node 1. For each node v, define its height h(v) as the number of edges on the longest downward path from v to a leaf in its subtree. A leaf has height 0. For any other node, h(v) = 1 + the maximum height among its children. The longest path whose highest node is v goes down through the two tallest child branches of v. If the two largest values of h(child) + 1 are best1 and best2 (taking 0 when a node has fewer than two children), then the longest path passing through v with v as its top node has best1 + best2 edges. The diameter is the maximum of this value over all nodes.

Why it is included. The method solves the problem in a single traversal and stores the information needed to reconstruct the path. It is a clear example of dynamic programming, because the answer for a node is built from stored answers for its children.

Relation to the tree. The subproblems are the subtrees. Each subtree height is calculated once and reused by the parent, so there is no repeated work.

Example. In the primary test case, h(6) = 1 and h(7) = 1, so at node 3 the two child values are 2 and 2, giving a path of 4 edges through node 3. At node 1, the child values are 2 (through node 2) and 3 (through node 3), giving 3 + 2 = 5 edges. This is the maximum, so the diameter is 5 edges.

Complexity. Time O(N), because each node is visited once and each edge is examined a constant number of times. Space O(N) for the adjacency list, the height information, and the recursion or traversal stack.

5.4 Module 4: Edmonds-Karp Maximum Flow

Purpose. This module computes the maximum flow from a source node to a destination node. Edmonds-Karp is an implementation of the Ford-Fulkerson method that always chooses the shortest augmenting path, found with Breadth First Search (BFS).

Working principle. The algorithm repeatedly searches the residual graph for the shortest path from the source to the destination along edges with remaining capacity. It pushes as much flow as the smallest residual capacity on that path allows, updates the residual capacities, and repeats until no such path exists. The total flow pushed is the maximum flow.

Model used in the website. The website uses a unit-capacity tree-edge model, in which every tree edge has capacity 1. The user supplies the source and destination as inputs. In this report, each undirected tree edge is understood as a pair of opposite arcs, each of capacity 1.

Why it is included. Network flow is a core topic of the course. Applying it to the tree shows how a general-purpose algorithm behaves on a special structure.

Relation to the tree. In a tree there is exactly one simple path between any two nodes. Under the unit-capacity model, this one path can carry exactly one unit of flow, so the maximum flow between two distinct nodes is 1. By the max-flow min-cut theorem, removing any single edge on the path separates the two nodes, which agrees with this value.

Example. With source 8 and destination 4, the shortest augmenting path is 8, 6, 3, 1, 2, 4. Its bottleneck capacity is 1, so one unit of flow is pushed. No second path exists, so the maximum flow is 1.

Complexity. For a general flow network with V vertices and E edges, Edmonds-Karp runs in O(V E^2) time. In the unit-capacity tree model, the flow value is at most 1, so at most one augmenting path is found, plus one final BFS that finds no path. Each BFS costs O(N), so the running time in this model is O(N). The space required is O(N).

5.5 Module 5: Vertex Cover 2-Approximation

Purpose. A vertex cover is a set of nodes such that every edge has at least one endpoint in the set. This module computes a vertex cover whose size is guaranteed to be at most twice the size of the smallest possible vertex cover.

Working principle. The algorithm builds a maximal matching, which is a set of edges that share no endpoints and to which no further edge can be added. It examines the edges one by one. If neither endpoint of an edge is already covered, the edge is selected and both of its endpoints are added to the cover. Every edge is then either selected or touches a covered node, so the result is a valid vertex cover.

Approximation guarantee. Let M be the selected matching. The edges in M share no endpoints, so any vertex cover must contain at least one separate endpoint for each of them. Hence the optimal cover has at least |M| nodes. The algorithm returns exactly 2|M| nodes, so its answer is at most twice the optimum.

Why it is included. Approximation algorithms are a topic of the course, and vertex cover is the standard example of a 2-approximation.

Relation to the tree. The edges of the tree form the input. For the primary test case, scanning the edges in the given order selects (1, 2), then (3, 6), then (7, 10), because every other edge already touches a covered node. The cover is {1, 2, 3, 6, 7, 10}, with 6 nodes. The smallest cover for this tree has 4 nodes, for instance {1, 2, 6, 7}, so the result is within the guarantee of 2 x 4 = 8.

Remark. For trees, the minimum vertex cover can in fact be computed exactly in linear time. The module is therefore intended to demonstrate the approximation technique, which becomes essential on general graphs, and not to replace an exact method. The chosen edges, and hence the cover, depend on the order in which edges are scanned.

Complexity. With one pass over the N - 1 edges and a marker for each node, the time is O(N) and the space is O(N).

5.6 Module 6: Randomized Quicksort

Purpose. This module sorts the node-degree sequence of the tree. The degree of a node is the number of edges attached to it.

Working principle. Quicksort chooses a pivot, partitions the array into elements not greater than the pivot and elements greater than the pivot, and sorts the two parts recursively. In randomised quicksort, the pivot is chosen at random. This removes the dependence of the running time on the original order of the input, so no particular input can consistently force poor behaviour.

Why it is included. Randomised algorithms are part of the course, and the degree sequence provides a natural list of numbers derived directly from the tree.

Relation to the tree. The degree of each node is the length of its adjacency list. For the primary test case, the degrees of nodes 1 to 10 are 2, 3, 3, 1, 1, 3, 2, 1, 1, 1. Their sum is 18, which equals 2 x 9, since every edge contributes to the degree of two nodes. Sorted in ascending order, the sequence is 1, 1, 1, 1, 1, 2, 2, 3, 3, 3.

Complexity. The expected time is O(N log N). The worst-case time is O(N^2), which occurs only if the random choices repeatedly give very unbalanced partitions, and this is extremely unlikely for large N. The expected extra space for the recursion stack is O(log N), and the worst case is O(N).

6. Practical Case Study Examples

This section illustrates the website with concrete inputs. All expected outputs were verified by hand using the method described in Section 13. In these examples, ties between equally long branches are resolved in favour of the child that appears first in the adjacency list, and the path is written starting from the end of the branch that has the larger height at the top node. A different tie-breaking rule can produce another path of the same length, and the diameter value itself does not change.

6.0 Primary Test Case

Number of nodes	10
Edges	1 2, 1 3, 2 4, 2 5, 3 6, 3 7, 6 8, 6 9, 7 10 (9 edges)
Tree diameter	5 edges
Diameter path	[8, 6, 3, 1, 2, 4], highlighted in red

This tree has other paths of the same length 5, for example [8, 6, 3, 1, 2, 5] and [10, 7, 3, 1, 2, 4]. The website displays one of them, namely [8, 6, 3, 1, 2, 4].

6.1 Case Study 1: Chain-Shaped Tree

A chain is a tree in which the nodes are connected in a single line.

Input	N = 5; edges 1 2, 2 3, 3 4, 4 5
Expected diameter	4 edges
Expected path	[5, 4, 3, 2, 1], which is the chain 1-2-3-4-5 read from the other end

Rooted at node 1, the heights are h(5) = 0, h(4) = 1, h(3) = 2, h(2) = 3, and h(1) = 4. Node 1 has only one child, so best1 = 4 and best2 = 0, giving 4 edges. The diameter of a chain with N nodes is always N - 1, and it is the worst case for tree height.

6.2 Case Study 2: Star-Shaped Tree

A star has one central node connected directly to all other nodes.

Input	N = 6; edges 1 2, 1 3, 1 4, 1 5, 1 6
Expected diameter	2 edges
Expected path	[2, 1, 3] (any two leaves joined through node 1 give a valid longest path)

Every leaf has height 0, so all child values at node 1 equal 1. The two largest are 1 and 1, giving 1 + 1 = 2 edges. The diameter of any star with at least three nodes is 2.

6.3 Case Study 3: Balanced Tree

A balanced tree has branches of nearly equal height. A complete binary tree with 7 nodes is used here.

Input	N = 7; edges 1 2, 1 3, 2 4, 2 5, 3 6, 3 7
Expected diameter	4 edges
Expected path	[4, 2, 1, 3, 6] (other paths such as [5, 2, 1, 3, 7] have the same length)

The heights are h(2) = h(3) = 1, so the child values at node 1 are 2 and 2, giving 4 edges. In a balanced tree the diameter passes through or near the root, and its length is about twice the height of the tree.

6.4 Case Study 4: KMP Pattern Search

Using the primary test case, the preorder sequence is [1, 2, 4, 5, 3, 6, 8, 9, 7, 10].

Pattern	Result	Explanation
[3, 6, 8]	Found at index 4	Elements at indices 4 to 6 are 3, 6, 8
[2, 4, 5]	Found at index 1	Elements at indices 1 to 3 are 2, 4, 5
[4, 3]	Not found	Node 4 is followed by node 5, not node 3

6.5 Case Study 5: Maximum-Flow Calculation

Input	Primary test case with source = 8 and destination = 4
Augmenting path	8, 6, 3, 1, 2, 4 (found by BFS)
Maximum flow	1 (unit capacity on each tree edge)

A second pair, source 8 and destination 10, uses the path 8, 6, 3, 7, 10 and also gives a maximum flow of 1. For any two distinct nodes of a tree under this model, the maximum flow is 1.

6.6 Case Study 6: Vertex-Cover Approximation

Input	Primary test case, edges scanned in the order given
Matching selected	(1, 2), (3, 6), (7, 10), so |M| = 3
Vertex cover returned	{1, 2, 3, 6, 7, 10}, size 6 = 2|M|
Optimal cover size	4, for example {1, 2, 6, 7}
Approximation ratio	6 / 4 = 1.5, which is within the guaranteed bound of 2

6.7 Case Study 7: Randomised Sorting of Node Degrees

Input degrees (nodes 1 to 10)	[2, 3, 3, 1, 1, 3, 2, 1, 1, 1]
Sorted output (ascending)	[1, 1, 1, 1, 1, 2, 2, 3, 3, 3]
Check	Sum of degrees = 18 = 2 x (number of edges)

The final sorted order is the same for every run, although the sequence of random pivot choices may differ from one run to another.

7. Expected Results and Benefits

The Tree Diameter Computation System is expected to provide the following benefits:

- **Visual learning:** Seeing the tree with the diameter in red helps students connect the numerical answer with the structure of the tree.
- **Automatic computation:** The user does not need to trace paths by hand. The diameter and path are produced immediately after the input is accepted.
- **Input validation:** Checks on the node count, edge count, and connectivity prevent meaningless results from invalid input.
- **Efficient processing:** The main algorithm runs in O(N) time, so even large trees are handled quickly in the browser.
- **Practical understanding of advanced algorithms:** Six related modules are applied to one tree, so students can compare how different techniques behave on the same data.
- **Complexity awareness:** Time and space complexity are displayed along with the results, which encourages the habit of analysing every algorithm.
- **Easy access:** Because the website is frontend-only, it needs no installation, server, or database and can be opened in a web browser.

These benefits depend on a correct implementation and on testing. The results should be checked against hand-calculated cases, such as those in Section 6, instead of being assumed correct.

8. Conclusion

The Tree Diameter Computation System is a frontend-only educational website built with HTML, CSS, and Vanilla JavaScript. It accepts a tree as a number of nodes and a list of edges, validates it, stores it as an adjacency list, computes the diameter using Tree Dynamic Programming with DFS in O(N) time and O(N) space, reconstructs the longest path, and draws the tree with the diameter highlighted in red. For the primary test case of 10 nodes and 9 edges, the diameter is 5 edges and the displayed path is [8, 6, 3, 1, 2, 4].

The project also demonstrates five other advanced-algorithm modules: algorithm analysis and complexity, KMP string matching on the DFS preorder sequence, Edmonds-Karp maximum flow under a unit-capacity tree-edge model, vertex cover 2-approximation using maximal matching, and randomised quicksort of the node-degree sequence.

Limitations

- A tree may have several longest paths, and the website displays only one of them.
- In the unit-capacity tree model, the maximum flow between two distinct nodes is always 1, so the flow module is mainly a demonstration of the method.
- The vertex cover module returns an approximate cover, not the minimum cover, although an exact method exists for trees.
- Because the website has no backend or database, entered trees are not stored permanently.
- Very large inputs are limited by the memory and speed of the user's browser and by the space available for drawing.

Possible Future Enhancements (Not Implemented)

- Step-by-step animation of the DFS and dynamic-programming process.
- Support for weighted edges, so that diameter can be measured as total weight.
- Importing trees from a file and exporting results or the drawing.
- Displaying all diameter paths instead of one.
- Comparing the running time of the O(N) method with a quadratic method on the same input.

PART B: ALGORITHM / PSEUDOCODE

9. Introduction to the Algorithmic Workflow

The Tree Diameter Computation System follows a fixed workflow. It accepts the number of nodes and the edges, validates them, builds an adjacency list, runs the main diameter algorithm, reconstructs the path, draws the tree, and then runs the additional algorithm modules. The results are displayed on the same page.

This part of the report describes each stage in a language-independent pseudocode. For each algorithm, the report gives its purpose, a step-by-step procedure, the pseudocode, a small example, the time and space complexity, and a short explanation. The examples use the primary test case of 10 nodes and 9 edges introduced in Section 6.

Note on pseudocode. The pseudocode in this report describes the standard form of each algorithm and the logic of the project as described in the project description. It is not a copy of the website source code. Before final submission, the team should compare each pseudocode block with the actual JavaScript implementation and adjust names or details where they differ (for example, the tie-breaking rule, the order of neighbours, and the partition scheme of quicksort).

Throughout this part, N is the number of nodes, E = N - 1 is the number of edges, adj[v] is the list of neighbours of node v, and nodes are numbered from 1 to N.

10. Overall Algorithm of the Website

Purpose. To control the complete flow from user input to displayed results.

Step-by-step procedure

1. Open the website and display the input area.
2. Accept the number of nodes N and the list of edges. For the relevant modules, also accept the numeric pattern (KMP) and the source and destination nodes (maximum flow).
3. Validate the input. If it is invalid, display an error message and stop.
4. Build the adjacency list.
5. Run Tree Dynamic Programming with DFS to compute the diameter.
6. Reconstruct the diameter path.
7. Draw the tree and highlight the diameter path in red.
8. Display the diameter, longest path, number of nodes, number of edges, time complexity, and space complexity.
9. Run and display the results of the KMP, Edmonds-Karp, vertex cover, and randomised quicksort modules.

Pseudocode

```
ALGORITHM TreeDiameterSystem_Main
BEGIN
  INPUT N, edgeList, pattern P, source s, destination t
  IF ValidateInput(N, edgeList) = FALSE THEN
    DISPLAY error message
    RETURN
  END IF
  adj <- BuildAdjacencyList(N, edgeList)
```

```
  (diameter, path) <- TreeDiameter(adj, N)
  DrawTree(N, edgeList, path)
  DISPLAY diameter, path, N, N-1, time and space complexity
  preorder <- BuildPreorder(adj, 1)
  matches  <- KMP_Search(preorder, P)
  flow     <- EdmondsKarp(adj, s, t)
  cover    <- VertexCover2Approx(edgeList, N)
  degrees  <- [ LENGTH(adj[v]) FOR v <- 1 TO N ]
  RandomizedQuickSort(degrees, 0, N-1)
  DISPLAY matches, flow, cover, degrees
END
```

Example. For the primary test case, the workflow validates 10 nodes and 9 edges, builds the adjacency list, finds a diameter of 5 and the path [8, 6, 3, 1, 2, 4], draws it, and then displays the results of the other modules.

Complexity. The total time is the sum of the stages. Each stage in this report is O(N) or O(N + M) except randomised quicksort, which is O(N log N) in expectation, so the overall expected time is O(N log N + M). The space is O(N + M).

Explanation. The main algorithm is a controller. It does not solve any problem by itself but calls each stage in order. Validation comes first, because every later stage assumes that the input is a proper tree. The order in which the modules run, or whether the user triggers them individually, should be described according to the website's actual behaviour.

11. Tree Input and Validation Algorithm

Purpose. To make sure that the entered data really describes a tree before any computation is done.

Input format. The nodes are numbered 1 to N. Each edge is a pair of node numbers u and v, written on a separate line or in an equivalent format. The edge text is split into numbers (parsed) and converted to integers.

Validation rules. A graph with N nodes is a tree if it is connected and has exactly N - 1 edges. The validation therefore checks:

- N is a positive integer.
- The number of edges is exactly N - 1.
- Each edge consists of two integers between 1 and N.
- All N nodes are connected, which is checked by traversing the graph from node 1 and counting the visited nodes.

A connected graph with N - 1 edges has no cycle. A self-loop, or a repeated edge, would use up one of the N - 1 edges without connecting a new node, so the graph would end up disconnected and be rejected by the connectivity check. Therefore these checks together are sufficient to confirm that the input is a tree.

Step-by-step procedure

1. Read N and check that it is a positive integer.
2. Parse the edge list into pairs of integers.
3. Compare the number of edges with N - 1.
4. Check that every node number lies between 1 and N.
5. Build a temporary adjacency list and traverse it from node 1.

6. If the number of visited nodes is not N, report that the graph is not connected.
7. If every check passes, accept the input.

Pseudocode

```
FUNCTION ValidateInput(N, edgeList)
  IF N is not an integer OR N < 1 THEN RETURN FALSE
  IF LENGTH(edgeList) != N - 1 THEN RETURN FALSE
  FOR each (u, v) IN edgeList DO
    IF u or v is not an integer in [1, N] THEN RETURN FALSE
  END FOR
  adj <- BuildAdjacencyList(N, edgeList)
  visited <- array of N+1 values, all FALSE
  count <- 0
  stack <- [1]; visited[1] <- TRUE
  WHILE stack is not empty DO
    x <- POP(stack); count <- count + 1
    FOR each y IN adj[x] DO
      IF visited[y] = FALSE THEN
        visited[y] <- TRUE; PUSH(stack, y)
  END WHILE
  RETURN (count = N)
END FUNCTION
```

Example. For the primary test case, N = 10 and there are 9 edges, all within 1 to 10, and the traversal from node 1 visits all 10 nodes, so the input is accepted. If N = 10 but only 8 edges are entered, the check fails at the second line. If N = 4 and the edges are 1 2, 1 2, 3 4, the count is correct, but the traversal from node 1 reaches only nodes 1 and 2, so the graph is reported as not connected.

Time complexity. O(N + E) = O(N). **Space complexity.** O(N).

Explanation. Counting edges is fast but cannot detect all errors, and connectivity alone cannot detect extra edges. Using both rules together, the validation guarantees that the structure is a tree.

12. Adjacency List Construction

Purpose. To store the tree in memory so that the neighbours of any node can be found quickly.

Working principle. An adjacency list keeps, for each node, a list of the nodes directly connected to it. Since tree edges are undirected, each edge (u, v) is stored twice: v is added to adj[u], and u is added to adj[v]. The total number of stored entries is therefore 2(N - 1). An adjacency matrix would need N x N cells, which is wasteful for a tree because a tree is sparse.

Step-by-step procedure

1. Create an empty list for each node from 1 to N.
2. For each edge (u, v), add v to the list of u.
3. Add u to the list of v.
4. Return the completed structure.

Pseudocode

```
FUNCTION BuildAdjacencyList(N, edgeList)
  FOR v <- 1 TO N DO
    adj[v] <- empty list
  END FOR
  FOR each (u, v) IN edgeList DO
    APPEND v TO adj[u]
```

```
    APPEND u TO adj[v]
  END FOR
  RETURN adj
END FUNCTION
```

Example. For the primary test case, the adjacency list is:

Node	Neighbours	Node	Neighbours
1	2, 3	6	3, 8, 9
2	1, 4, 5	7	3, 10
3	1, 6, 7	8	6
4	2	9	6
5	2	10	7

The total number of entries is 18, which equals 2 x 9. The degree of node 3 is 3, because three neighbours are listed.

Time complexity. O(N + E) = O(N). **Space complexity.** O(N), because only 2(N - 1) entries and N lists are stored.

Explanation. The order in which neighbours are listed follows the order of the edges in the input. This order is later used by the DFS, by the preorder sequence for KMP, and for tie-breaking.

13. DFS and Tree Dynamic Programming

Purpose. To compute the diameter of the tree in a single traversal and to record the information needed to reconstruct the longest path.

Working principle. The DFS starts at the root, node 1, and moves to each unvisited neighbour. Because the graph is a tree, the only neighbour that has already been visited is the parent, so the parent is passed along and skipped. When the DFS returns from a child u to its parent v, it brings back the height of u. The parent then computes the following:

- The value h(u) + 1 for each child u, which is the number of edges from v down to the deepest leaf through u.
- The two largest of these values, best1 and best2 (0 if v has fewer than two children).
- The longest path with v as its highest node, which has best1 + best2 edges.
- The height of v, which is best1.

The diameter is the largest best1 + best2 over all nodes. Any longest path in a rooted tree has a unique highest node, and at this node the path descends into two different child branches (or one branch, or none). Since the algorithm tries every node as the highest node and uses the tallest branches, it cannot miss the longest path.

Step-by-step procedure

1. Set the best diameter so far to -1 and the best top node to none.
2. Start DFS from node 1 with no parent.
3. At each node, set best1 and best2 to 0 and the two best children to none.
4. For every neighbour that is not the parent, call DFS and add 1 to the returned height.

5. Update best1 and best2 and the corresponding children. A new value replaces best1 only if it is strictly greater, so on ties the earlier child is kept.
6. If best1 + best2 is greater than the best diameter so far, store it together with the node and its two best children.
7. Store the best child of the node (the child giving best1) so the path can be followed later, and return best1.

Pseudocode

```
ALGORITHM TreeDiameter(adj, N)
  diameter <- -1; topNode <- NONE; topChild1 <- NONE; topChild2 <- NONE
  CALL DFS(1, NONE)
  RETURN diameter

FUNCTION DFS(v, parent)
  best1 <- 0; best2 <- 0; child1 <- NONE; child2 <- NONE
  FOR each u IN adj[v] DO
    IF u != parent THEN
      h <- DFS(u, v) + 1
      IF h > best1 THEN
        best2 <- best1; child2 <- child1
        best1 <- h;     child1 <- u
      ELSE IF h > best2 THEN
        best2 <- h;     child2 <- u
      END IF
    END IF
  END FOR
  IF best1 + best2 > diameter THEN
    diameter <- best1 + best2
    topNode <- v; topChild1 <- child1; topChild2 <- child2
  END IF
  nextDown[v] <- child1
  RETURN best1
END FUNCTION
```

Example (primary test case). The traversal order is 1, 2, 4, 5, 3, 6, 8, 9, 7, 10, and the nodes finish in the order 4, 5, 2, 8, 9, 6, 10, 7, 3, 1. The values computed are as follows.

Node	Children	Child values (h + 1)	h(v)	Longest path with v on top
4, 5, 8, 9, 10	None (leaves)	-	0	0
2	4, 5	1, 1	1	1 + 1 = 2
6	8, 9	1, 1	1	1 + 1 = 2
7	10	1	1	1 + 0 = 1
3	6, 7	2, 2	2	2 + 2 = 4
1	2, 3	2, 3	3	3 + 2 = 5

The maximum value is 5, obtained at node 1, so **the tree diameter is 5 edges**. At node 1, best1 = 3 comes from child 3 and best2 = 2 comes from child 2.

Time complexity. O(N). Each node is entered once, and each adjacency entry is examined once, which gives 2(N - 1) examinations in total. **Space complexity.** O(N) for the adjacency list and the stored arrays. The recursion stack is as deep as the height of the tree, which is O(N) in the worst case (for a chain).

Explanation. This is dynamic programming because the height of a node is defined in terms of the heights of its children, each of which is computed once and then reused. If the implementation is recursive, a very deep chain may reach the browser's call-stack limit. An implementation that uses an explicit stack avoids this problem, and the team should state which one is used.

14. Diameter Path Reconstruction

Purpose. To obtain the actual sequence of nodes on the longest path, not only its length.

Working principle. The DFS stored the top node of the longest path and its two best children. From each of these children, the path continues downward by repeatedly moving to the stored best child, until a node with no child is reached. That final node is an end point of the diameter. The two downward branches are joined at the top node: the first branch is reversed so that the path reads from one end point, through the top node, to the other end point.

Step-by-step procedure

1. Start with an empty list left and move to the first best child of the top node.
2. While the current node exists, append it to left and move to its stored best child.
3. Repeat the same process from the second best child to build the list right.
4. Reverse left.
5. Form the path as reversed left, then the top node, then right.
6. Check that the number of edges, which is the number of nodes minus 1, equals the diameter.

Pseudocode

```
FUNCTION ReconstructPath(topNode, topChild1, topChild2, nextDown)
  left <- empty list; cur <- topChild1
  WHILE cur != NONE DO
    APPEND cur TO left; cur <- nextDown[cur]
  END WHILE
  right <- empty list; cur <- topChild2
  WHILE cur != NONE DO
    APPEND cur TO right; cur <- nextDown[cur]
  END WHILE
  RETURN REVERSE(left) + [topNode] + right
END FUNCTION
```

Example. In the primary test case, topNode = 1, topChild1 = 3, and topChild2 = 2. Following the best children from node 3 gives 3, 6, 8, so left = [3, 6, 8]. From node 2 the process gives 2, 4, so right = [2, 4]. The path is [8, 6, 3] + [1] + [2, 4] = **[8, 6, 3, 1, 2, 4]**. It has 6 nodes and 5 edges, which agrees with the diameter. The end points are the leaves 8 and 4.

Node 6 has two children of equal height, 8 and 9, and node 2 has two, 4 and 5. Because a strictly greater value is needed to replace the stored child, the first one (8 and 4 respectively) is kept. This tie-breaking rule explains why the path ends at 8 and 4 and not at 9 or 5.

Time complexity. O(D), where D is the number of edges on the diameter, so O(N) in the worst case. **Space complexity.** O(D), which is O(N) in the worst case.

Explanation. Storing only one best child per node is enough to rebuild the path, and no second traversal of the tree is needed.

15. Tree Visualisation

Purpose. To draw the tree on the webpage and highlight the diameter path in red.

Working principle. Each node is assigned a position on the drawing area. Each edge is drawn as a line between the positions of its two endpoints, and each node is drawn as a labelled circle. To highlight the diameter, the consecutive pairs of nodes in the path are converted to a set of edges. Any edge in this set is drawn in red, and all other edges are drawn in the normal colour.

Step-by-step procedure

1. Compute a position for every node (the layout).
2. Create a set of highlighted edges from consecutive nodes in the diameter path.
3. Draw every edge, using red for edges in the highlighted set.
4. Draw every node and its label on top of the edges.
5. Show the diameter, path, node count, edge count, and complexity information beside the drawing.

Pseudocode

```
PROCEDURE DrawTree(N, edgeList, path)
  positions <- ComputeLayout(N, edgeList)
  highlight <- empty set
  FOR i <- 0 TO LENGTH(path) - 2 DO
    ADD edge(path[i], path[i+1]) TO highlight
  END FOR
  FOR each (u, v) IN edgeList DO
    IF edge(u, v) IN highlight THEN colour <- RED ELSE colour <- NORMAL
    DRAW line from positions[u] to positions[v] USING colour
  END FOR
  FOR v <- 1 TO N DO
    DRAW circle at positions[v] AND label v
  END FOR
END PROCEDURE
```

Example. For the path [8, 6, 3, 1, 2, 4], the highlighted edges are 8-6, 6-3, 3-1, 1-2, and 2-4. The remaining four edges, 2-5, 3-7, 6-9, and 7-10, are drawn in the normal colour.

Time complexity. O(N) drawing operations for N nodes and N - 1 edges, plus the cost of the layout. **Space complexity.** O(N) for the positions and the highlighted set.

To be completed by the team: The layout method and the drawing technique (for example, SVG elements or the canvas element) are implementation details that are not specified in the project description. Add a sentence stating which are used in the website.

Explanation. Drawing is separate from the computation. The visualisation only reads the result of the diameter algorithm, so any change in the layout does not affect the correctness of the diameter.

16. KMP String Matching Algorithm

Purpose. To find all positions at which a numeric pattern occurs in the DFS preorder sequence of the tree.

Working principle. KMP uses the failure table lps[], where lps[i] is the length of the longest proper prefix of the pattern P[0..i] that is also a suffix of it. If a mismatch occurs after j

characters have matched, the pattern pointer moves back to lps[j - 1] while the text pointer stays where it is. As a result, each text element is compared a bounded number of times.

Step-by-step procedure

1. Build the preorder sequence T of the tree by a DFS from node 1.
2. Build the failure table of the pattern P.
3. Compare T[i] with P[j]. If they are equal, advance both pointers.
4. If j reaches the pattern length M, record a match at index i - M and set j to lps[M - 1].
5. On a mismatch, if j > 0 set j to lps[j - 1]; otherwise advance i.
6. Continue until the whole sequence is scanned and display all recorded matches.

Pseudocode

```
PROCEDURE BuildPreorder(v, parent, T)
  APPEND v TO T
  FOR each u IN adj[v] DO
    IF u != parent THEN BuildPreorder(u, v, T)
END PROCEDURE

FUNCTION BuildLPS(P, M)
  lps[0] <- 0; len <- 0; i <- 1
  WHILE i < M DO
    IF P[i] = P[len] THEN
      len <- len + 1; lps[i] <- len; i <- i + 1
    ELSE IF len != 0 THEN
      len <- lps[len - 1]
    ELSE
      lps[i] <- 0; i <- i + 1
    END IF
  END WHILE
  RETURN lps
END FUNCTION

FUNCTION KMP_Search(T, N, P, M)
  lps <- BuildLPS(P, M); matches <- empty list
  i <- 0; j <- 0
  WHILE i < N DO
    IF T[i] = P[j] THEN
      i <- i + 1; j <- j + 1
      IF j = M THEN
        APPEND (i - j) TO matches; j <- lps[j - 1]
      END IF
    ELSE IF j != 0 THEN
      j <- lps[j - 1]
    ELSE
      i <- i + 1
    END IF
  END WHILE
  RETURN matches
END FUNCTION
```

Example. For the primary test case, T = [1, 2, 4, 5, 3, 6, 8, 9, 7, 10]. For P = [3, 6, 8], the failure table is [0, 0, 0], because no element repeats. The pointers advance without any fallback until T[4] = 3, T[5] = 6, and T[6] = 8 match the whole pattern, so a match is recorded at index 4. As an illustration of the failure table, the pattern [1, 2, 1, 2, 3] has lps = [0, 0, 1, 2, 0]. This pattern does not occur in a tree preorder, because node numbers do not repeat, but it shows how the table is built.

Time complexity. Building the preorder takes O(N), building the failure table O(M), and searching O(N). The worst-case total is O(N + M). **Space complexity.** O(N) for the sequence and O(M) for the failure table, so O(N + M) in total.

Explanation. The text pointer never moves backwards, which is why the running time is linear. Since the preorder sequence depends on the root and the neighbour order, the same tree entered with a different edge order may give a different sequence and different matches.

17. Edmonds-Karp Maximum-Flow Algorithm

Purpose. To compute the maximum flow from the source s to the destination t in the tree, using the unit-capacity tree-edge model.

Working principle. The algorithm keeps a residual capacity for every arc. It repeatedly finds the shortest path from s to t in the residual graph using BFS, computes the smallest residual capacity (the bottleneck) on that path, subtracts it from the arcs along the path, and adds it to the reverse arcs. When BFS can no longer reach t, the accumulated flow is the maximum. In this model, each tree edge has capacity 1.

Step-by-step procedure

1. Set the capacity of each tree edge to 1 and the flow to 0.
2. Run BFS from s over arcs with positive residual capacity, recording each node's parent.
3. If t was not reached, stop and return the flow.
4. Trace the path from t back to s using the parents and find the bottleneck capacity.
5. Update the residual capacities along the path and add the bottleneck to the flow.
6. Return to step 2.

Pseudocode

```
FUNCTION EdmondsKarp(adj, s, t)
  FOR each tree edge (u, v) DO res(u, v) <- 1; res(v, u) <- 1
  maxFlow <- 0
  LOOP
    parent[1..N] <- NONE; parent[s] <- s; queue <- [s]
    WHILE queue not empty AND parent[t] = NONE DO
      x <- DEQUEUE(queue)
      FOR each y IN adj[x] DO
        IF parent[y] = NONE AND res(x, y) > 0 THEN
          parent[y] <- x; ENQUEUE(queue, y)
      END FOR
    END WHILE
    IF parent[t] = NONE THEN EXIT LOOP
    b <- INFINITY; v <- t
    WHILE v != s DO
      b <- MIN(b, res(parent[v], v)); v <- parent[v]
    END WHILE
    v <- t
    WHILE v != s DO
      res(parent[v], v) <- res(parent[v], v) - b
      res(v, parent[v]) <- res(v, parent[v]) + b
      v <- parent[v]
    END WHILE
    maxFlow <- maxFlow + b
  END LOOP
  RETURN maxFlow
END FUNCTION
```

Example. In the primary test case with s = 8 and t = 4, the first BFS reaches node 4 through the parents 4 <- 2 <- 1 <- 3 <- 6 <- 8. The bottleneck is 1, so the flow becomes 1. The residual capacity of each arc on the path in the direction from 8 to 4 becomes 0. The second BFS cannot reach node 4, so the algorithm stops and returns 1.

Time complexity. For a general network, O(V E^2). In this model the flow is at most 1, so at most one augmentation takes place, and each BFS costs O(N + E) = O(N), so the time is O(N). **Space complexity.** O(N) for the residual capacities, parent array, and queue.

Explanation. Choosing the shortest path with BFS is what distinguishes Edmonds-Karp from the general Ford-Fulkerson method, and it is the reason for the polynomial bound. On a tree, the single path between s and t means that the answer is always 1 for distinct nodes. The website's handling of the case s = t should be stated by the team if it is shown.

18. Vertex Cover 2-Approximation Algorithm

Purpose. To compute a vertex cover that is at most twice the size of the minimum vertex cover.

Working principle. The algorithm builds a maximal matching greedily. It scans the edges once. When both endpoints of an edge are not yet covered, the edge is added to the matching and both endpoints are added to the cover. Every edge is either in the matching, in which case both of its endpoints are in the cover, or it was skipped because an endpoint was already covered. Hence all edges are covered. Since the matching edges are disjoint, the minimum cover has at least |M| nodes, while the algorithm returns 2|M|.

Step-by-step procedure

1. Mark all nodes as not covered and start with an empty cover and matching.
2. Take the next edge (u, v).
3. If neither u nor v is covered, add the edge to the matching, mark both nodes covered, and add both to the cover.
4. Otherwise skip the edge.
5. After all edges are scanned, return the cover and the matching.

Pseudocode

```
FUNCTION VertexCover2Approx(edgeList, N)
  covered[1..N] <- FALSE
  cover <- empty list; matching <- empty list
  FOR each (u, v) IN edgeList DO
    IF covered[u] = FALSE AND covered[v] = FALSE THEN
      APPEND (u, v) TO matching
      covered[u] <- TRUE; covered[v] <- TRUE
      APPEND u TO cover; APPEND v TO cover
    END IF
  END FOR
  RETURN cover, matching
END FUNCTION
```

Example. For the primary test case, the edges are scanned in the order given. The edge (1, 2) is selected. Edges (1, 3), (2, 4), and (2, 5) are skipped. The edge (3, 6) is selected, and (3, 7), (6, 8), and (6, 9) are skipped. The edge (7, 10) is selected. The matching is {(1, 2), (3, 6), (7, 10)}, and the cover is {1, 2, 3, 6, 7, 10} with 6 nodes. The optimal cover {1, 2, 6, 7} has 4 nodes, so 6 is not more than 2 x 4.

Time complexity. O(N), because there are N - 1 edges and each is handled in constant time. **Space complexity.** O(N) for the covered markers and the output lists.

Explanation. The guarantee holds for any order of the edges, although the particular cover returned depends on the order. On trees, an exact minimum cover can be found efficiently, so this module demonstrates an idea that matters mainly for general graphs.

19. Randomized Quicksort Algorithm

Purpose. To sort the node-degree sequence using quicksort with a randomly chosen pivot.

Working principle. A pivot is chosen uniformly at random from the current range and moved to the end of the range. The range is then partitioned so that elements not greater than the pivot come before it and larger elements come after it. The pivot is placed in its final position, and the two sides are sorted recursively. Since each element is equally likely to be chosen as the pivot, the partitions are balanced on average, whatever the initial order of the input.

Step-by-step procedure

1. Compute the degree of every node as the length of its adjacency list.
2. If the range has fewer than two elements, stop.
3. Choose a random index in the range and swap that element with the last element.
4. Partition the range around the pivot.
5. Recursively sort the left part and the right part.
6. Display the sorted degree sequence.

Pseudocode

```
PROCEDURE RandomizedQuickSort(A, low, high)
  IF low < high THEN
    p <- RandomizedPartition(A, low, high)
    RandomizedQuickSort(A, low, p - 1)
    RandomizedQuickSort(A, p + 1, high)
  END IF
END PROCEDURE

FUNCTION RandomizedPartition(A, low, high)
  r <- random integer in [low, high]
  SWAP A[r] AND A[high]
  pivot <- A[high]; i <- low - 1
  FOR j <- low TO high - 1 DO
    IF A[j] <= pivot THEN
      i <- i + 1; SWAP A[i] AND A[j]
    END IF
  END FOR
  SWAP A[i + 1] AND A[high]
  RETURN i + 1
END FUNCTION
```

Example. The degrees of the primary test case are A = [2, 3, 3, 1, 1, 3, 2, 1, 1, 1]. Suppose the random index chosen first is 6, whose value is 2. After swapping it with the last element, partitioning gives [2, 1, 1, 1, 1, 1, 2, 3, 3, 3], with the pivot 2 fixed at index 6. The left part [2, 1, 1, 1, 1, 1] and the right part [3, 3, 3] are sorted recursively. The final result is [1, 1, 1, 1, 1, 2, 2, 3, 3, 3]. The random choice differs from run to run, but the sorted result is always the same.

Time complexity. Expected O(N log N); worst case O(N^2). **Space complexity.** Sorting is done in place. The recursion stack has expected depth O(log N) and worst-case depth O(N).

Explanation. The worst case occurs only when the random pivots are repeatedly the smallest or largest elements, which has a very small probability for large N. The partition scheme shown here is the common one and should be compared with the website's implementation. This scheme can partition unevenly when there are many equal values, as in a star tree where almost all degrees equal 1, so a three-way partition is a possible improvement.

20. Complexity and Algorithm Analysis

Algorithm analysis describes how the time and memory needs of an algorithm grow with the input size. The following variables are used: N is the number of nodes, E = N - 1 is the number of edges, M is the length of the KMP pattern, D is the number of edges on the diameter path, and V and E in the general maximum-flow bound refer to the number of vertices and edges of a flow network.

Algorithm	Purpose	Time Complexity	Space Complexity
Input validation	Check that the input is a tree	O(N)	O(N)
Adjacency-list construction	Store the tree in memory	O(N)	O(N)
Tree DP with DFS (main algorithm)	Compute the tree diameter	O(N)	O(N)
Path reconstruction	Recover the diameter path	O(D), at most O(N)	O(D), at most O(N)
Tree visualisation	Draw the tree and highlight the path	O(N) drawing operations plus layout	O(N)
KMP string matching	Find a pattern in the preorder sequence	O(N + M) worst case	O(N + M)
Edmonds-Karp maximum flow	Maximum flow from source to destination	General: O(V E^2); this model: O(N)	O(N)
Vertex cover 2-approximation	Find a cover at most twice the optimum	O(N)	O(N)
Randomized quicksort	Sort the node degrees	Expected O(N log N); worst case O(N^2)	Expected O(log N) stack; worst case O(N)

20.1 Why the Main Algorithm Takes O(N) Time

The DFS enters each node exactly once. While processing a node, it examines every entry in that node's adjacency list once. The sum of the adjacency-list lengths is 2(N - 1), so the total number of examinations is 2(N - 1). At each node, the update of best1 and best2 and the comparison with the stored diameter take constant time. The total work is therefore proportional to N + 2(N - 1), which is O(N).

20.2 Why the Main Algorithm Takes O(N) Space

The adjacency list stores 2(N - 1) entries and N list headers. The height and next-child information needs one value per node. The recursion stack holds one frame per node on the current root-to-node path, which is at most N frames for a chain. All of these are O(N), so the total is O(N).

20.3 Worst-Case and Expected Complexity

Most algorithms in the table have the same time in every case, because they scan the input a fixed number of times. KMP is linear in the worst case, and this holds for any pattern and any sequence. Edmonds-Karp has a worst-case bound of O(V E^2) for general networks, but the tree model with unit capacities needs at most one augmentation. Randomised quicksort is the only algorithm where the distinction is essential: its O(N log N) time is an expected value over the random choices of the algorithm, and the O(N^2) worst case remains possible but very unlikely.

20.4 Comparison with a Simple Approach

Method	Idea	Time	Space
All-pairs traversal	Run a traversal from every node and take the largest distance	O(N^2)	O(N)
Tree DP with DFS (used)	One traversal using subtree heights	O(N)	O(N)

For N = 100,000 the simple method needs about 10^10 steps, while the linear method needs about 10^5, so the dynamic-programming approach is much better suited to large inputs.

21. Pseudocode Conclusion

Part B described the workflow of the Tree Diameter Computation System and the algorithms that it uses. The website validates the tree, represents it as an adjacency list, computes the diameter with Tree Dynamic Programming and DFS in O(N) time and O(N) space, reconstructs the path [8, 6, 3, 1, 2, 4] for the primary test case, and draws the tree with the diameter in red.

The remaining modules apply other advanced techniques to the same tree. KMP searches the preorder sequence in O(N + M) time. Edmonds-Karp finds a maximum flow of 1 between two nodes in the unit-capacity model. The vertex cover algorithm returns a valid cover within twice the optimum. Randomised quicksort sorts the degree sequence in expected O(N log N) time.

The pseudocode is intended as a clear description of the logic and as a guide for the implementation. As noted in Section 9, the team should verify each block against the actual source code before submission, since the final behaviour depends on the implementation and on testing with the cases in Section 6.

PART C: RESULTS

22. Results and Website Screenshots

This section presents the output of the Tree Diameter Computation System. All figures use the primary test case (10 nodes and the edges 1 2, 1 3, 2 4, 2 5, 3 6, 3 7, 6 8, 6 9, 7 10) unless stated otherwise. The expected diameter is 5 edges and the expected path is [8, 6, 3, 1, 2, 4], highlighted in red.

Instruction for the team: The boxes below are placeholders only. Replace each one with a real screenshot taken from your running website, using the input stated in the caption, and delete the placeholder text. Check that each screenshot agrees with the description under it, and edit the description if the website shows something different.

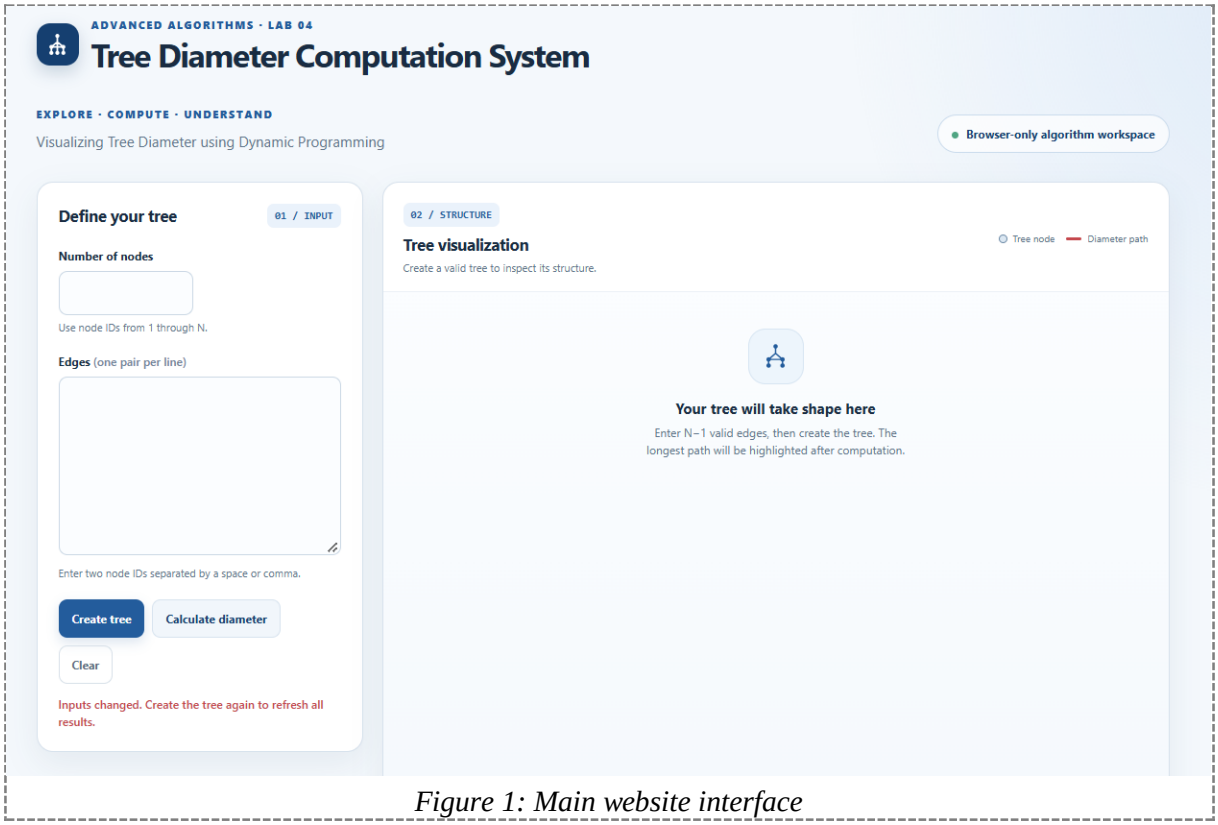


Figure 1: Main website interface

Figure 1: Main interface of the Tree Diameter Computation System

This screenshot should show the complete landing view of the website, including the title, the input area, and the module sections. It demonstrates that the system is a single-page, frontend-only application that runs in the browser.

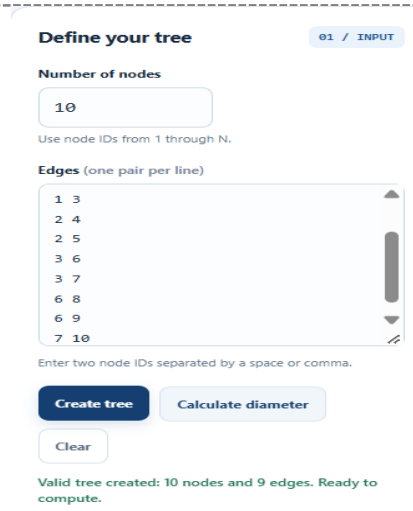


Figure 2: Tree input section

Figure 2: Tree input section with 10 nodes and 9 edges

This screenshot should show the number of nodes entered as 10 and the nine edges entered in the input box. It demonstrates how the user supplies the tree and where the input is validated before any computation.

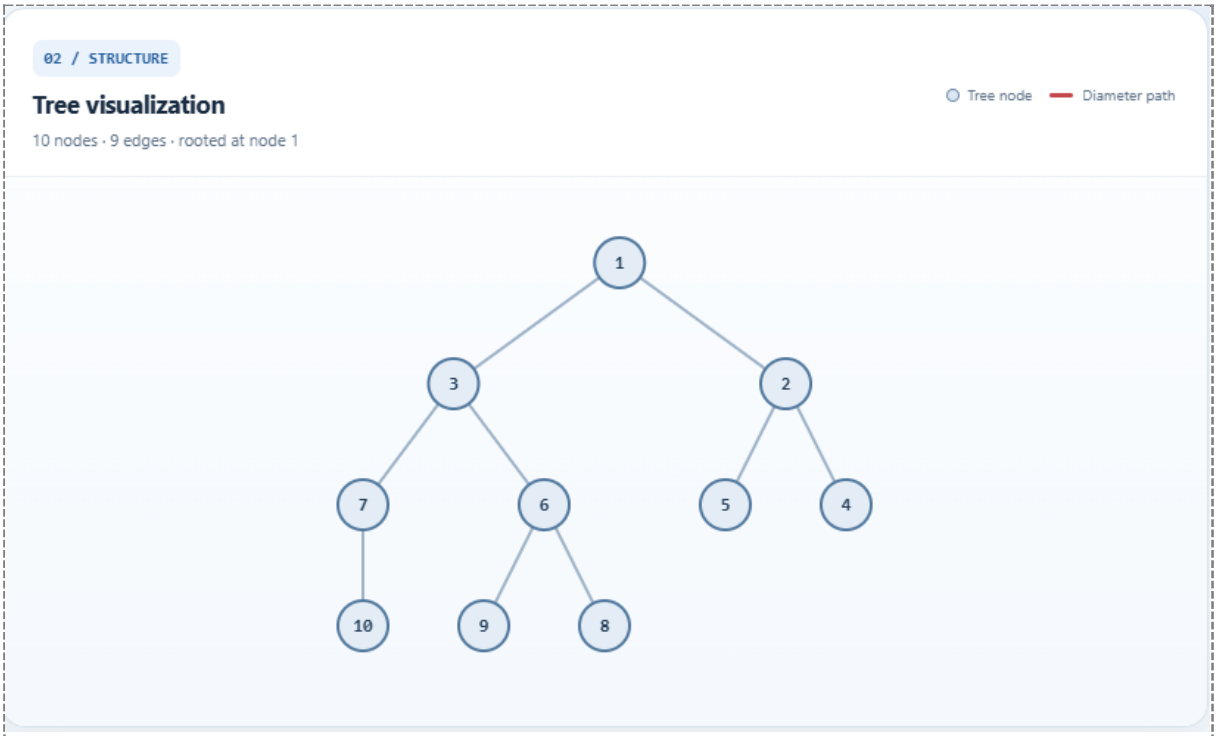


Figure 3: Tree visualisation

Figure 3: Tree visualisation with the diameter path in red

This screenshot should show the drawn tree with nodes as labelled circles. The edges 8-6, 6-3, 3-1, 1-2, and 2-4 should appear in red, and the other edges in the normal colour. It demonstrates how the longest path can be identified visually.

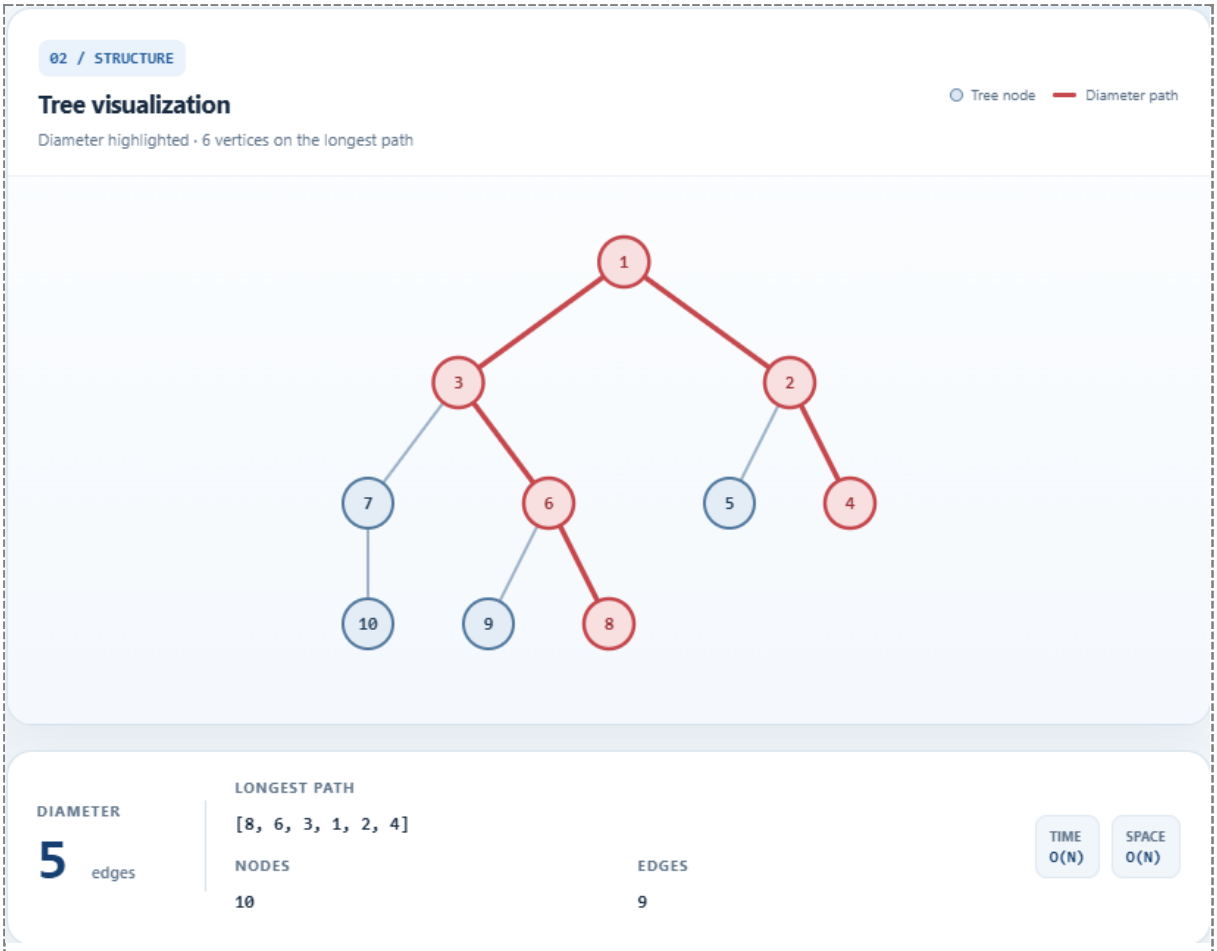


Figure 4: Diameter and longest-path output

Figure 4: Diameter, longest path, and tree information output

This screenshot should show the diameter as 5 edges and the longest path as [8, 6, 3, 1, 2, 4]. It should also show 10 nodes, 9 edges, the time complexity O(N), and the space complexity O(N). It demonstrates that the numerical results agree with the hand-calculated values in Section 13.

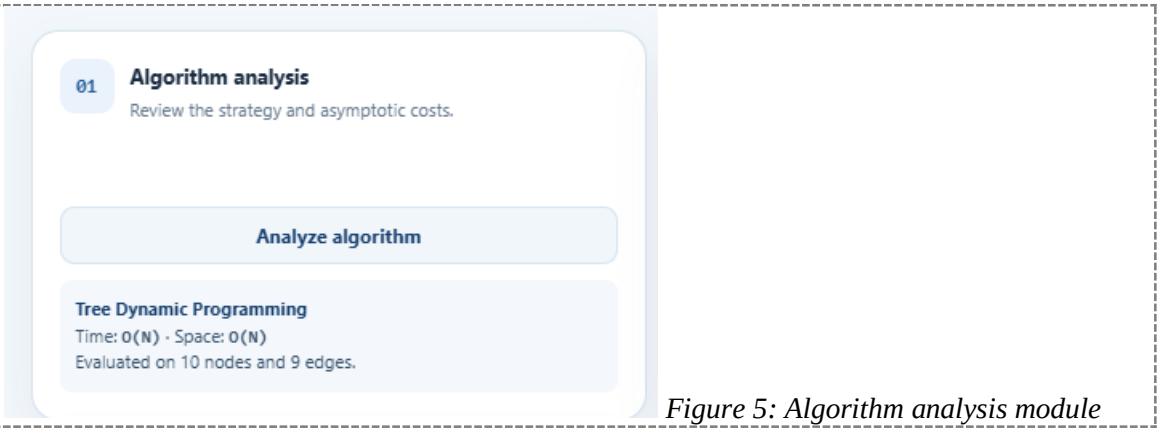


Figure 5: Algorithm analysis module

Figure 5: Algorithm Analysis and Complexity module

This screenshot should show the module that explains the choice of algorithm and its asymptotic time and space complexity. It demonstrates how the website links the displayed result to the cost of computing it.

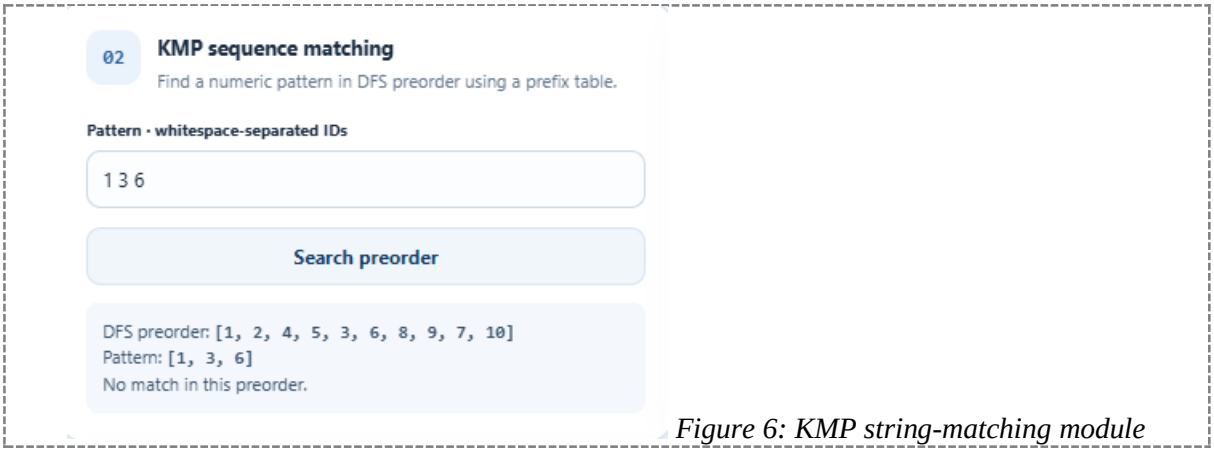


Figure 6: KMP string-matching module

Figure 6: KMP string-matching module on the DFS preorder sequence

This screenshot should show the preorder sequence [1, 2, 4, 5, 3, 6, 8, 9, 7, 10] (assuming neighbours are visited in input order) and a pattern such as [3, 6, 8] with the reported match at index 4. It demonstrates a linear-time pattern search on a sequence derived from the tree.

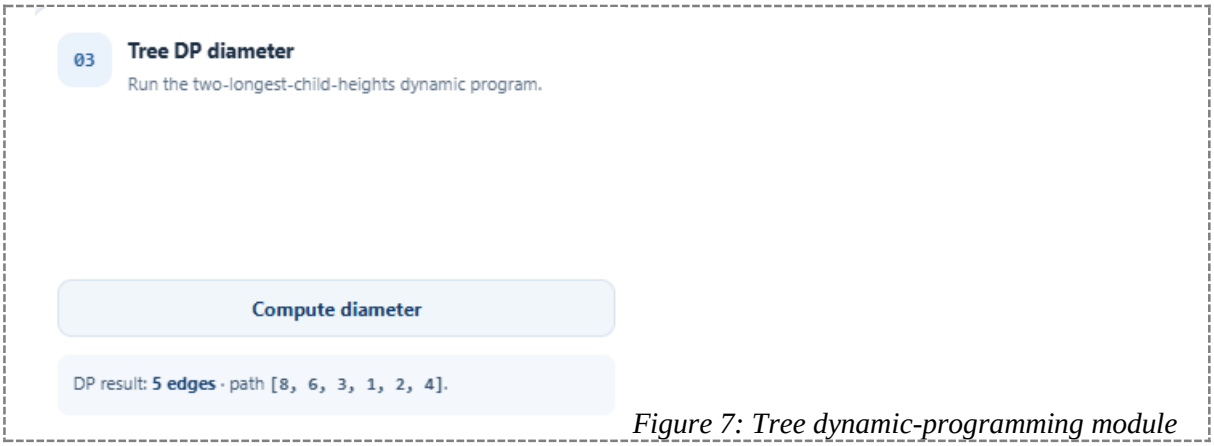


Figure 7: Tree dynamic-programming module

Figure 7: Tree dynamic-programming module showing subtree heights

This screenshot should show the main algorithm module with the subtree heights and the two largest child heights, resulting in a diameter of 5 at node 1. It demonstrates how the diameter is built from the heights of subtrees.

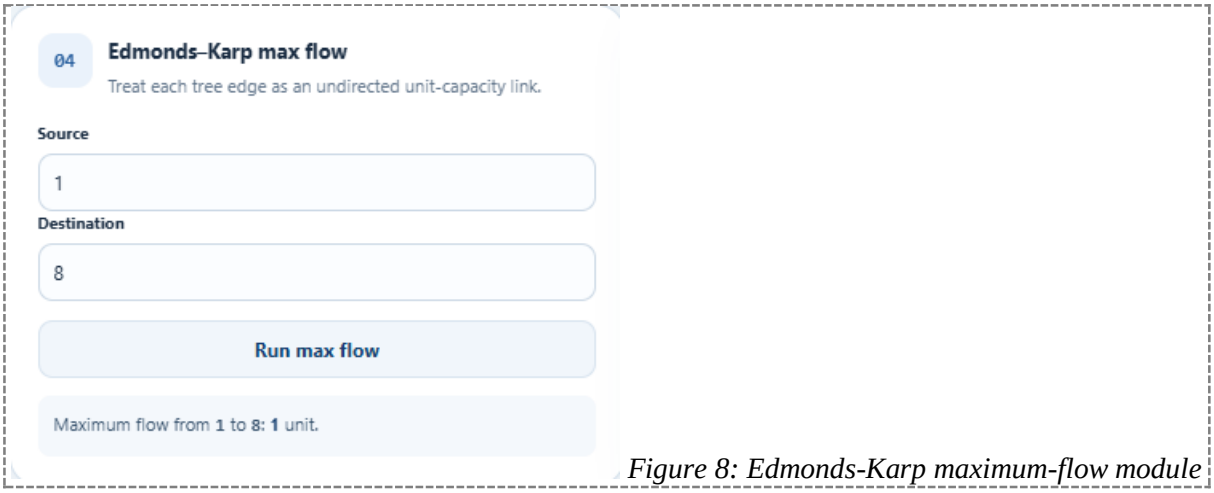


Figure 8: Edmonds-Karp maximum-flow module

Figure 8: Edmonds-Karp maximum-flow module with source 8 and destination 4

This screenshot should show the source and destination inputs, the augmenting path found, and the maximum flow of 1. It demonstrates the unit-capacity tree-edge model, in which a tree has a single path between two nodes.

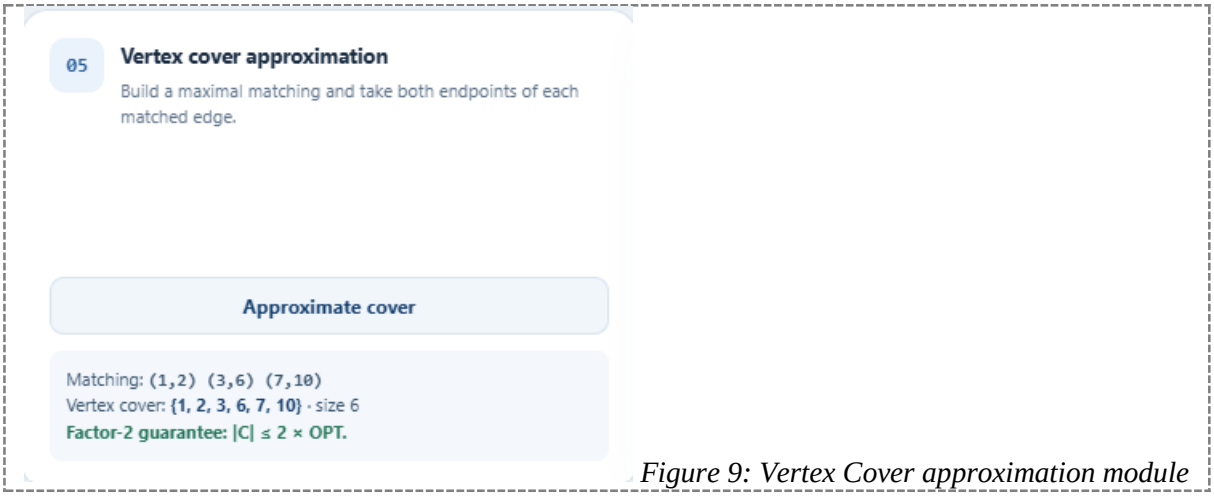


Figure 9: Vertex Cover approximation module

Figure 9: Vertex Cover 2-approximation module

This screenshot should show the selected matching and the resulting vertex cover. With edges scanned in the given order, these are the matching (1, 2), (3, 6), (7, 10) and the cover {1, 2, 3, 6, 7, 10}. It demonstrates the factor-2 guarantee, since the cover has twice as many nodes as the matching.

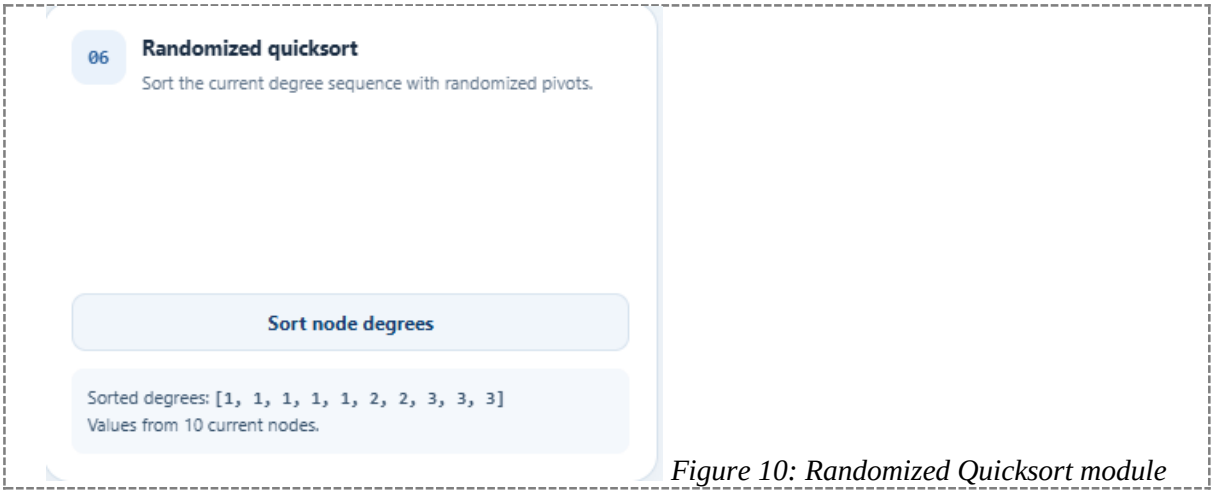


Figure 10: Randomized Quicksort module

Figure 10: Randomized Quicksort module sorting the node degrees

This screenshot should show the unsorted degree sequence [2, 3, 3, 1, 1, 3, 2, 1, 1, 1] and the sorted result [1, 1, 1, 1, 1, 2, 2, 3, 3, 3]. It demonstrates the use of random pivot selection on data taken from the tree.

22.1 Summary of Verified Results

Test case	Nodes	Edges	Diameter	Path
Primary test case	10	9	5	[8, 6, 3, 1, 2, 4]
Chain	5	4	4	[5, 4, 3, 2, 1]
Star	6	5	2	[2, 1, 3]
Balanced (complete binary)	7	6	4	[4, 2, 1, 3, 6]

For trees with several longest paths, the path shown depends on the tie-breaking rule. The values of the diameter are independent of this rule.

23. Final Conclusion

The Tree Diameter Computation System shows how a tree problem can be solved, visualised, and used to demonstrate several advanced algorithms in a browser-based tool built only with HTML, CSS, and Vanilla JavaScript. The system validates the number of nodes and edges, checks connectivity, builds an adjacency list, and computes the diameter in O(N) time and O(N) space using Tree Dynamic Programming with DFS. For the primary test case, it finds a diameter of 5 edges with the path [8, 6, 3, 1, 2, 4], drawn in red on the tree.

The six modules link the project to the course. Algorithm analysis justifies the choice of the linear method. KMP searches the preorder sequence in O(N + M) time. Edmonds-Karp computes a maximum flow, which equals 1 between two distinct nodes in the unit-capacity tree model. The vertex cover algorithm returns a cover within twice the optimum size. Randomised quicksort sorts the degree sequence in expected O(N log N) time.

The project has some limitations. It displays only one of the possible longest paths, it does not store data, and its flow and vertex cover modules are demonstrations, because exact and simpler methods exist for trees. Possible future enhancements, which are not part of the current implementation, include step-by-step animation, weighted edges, file import and export, and

listing all diameter paths. In summary, the project meets its objectives of efficient computation, clear visualisation, and practical learning of advanced algorithms.